

图社区发现（L6）完全总结

社区发现的目的是在网络中找出紧密连接的节点群组。通俗来说就是"物以类聚，人以群分"——把网络中联系紧密的节点聚成一个个社区。

一、背景与基本概念

1.1 网络与社区

- 社区（Community）：网络中紧密相连的节点集合
- 模块 $\leq$ 簇 $\leq$ 社区（从属关系）
- 社区发现：基于链接拓扑结构发现社区（不同于聚类）

1.2 非重叠 vs 重叠社区

类型	说明
非重叠	每个节点只属于一个社区
重叠	节点可属于多个社区

1.3 应用场景

- 赞助搜索：找微市场
- 电影-演员聚类
- 社交网络：发现朋友圈/信任圈
- 组织结构分析

二、什么是"好"的社区？

2.1 分割质量度量

目标：最大化社区内部连接，最小化社区间连接。

2.2 传导率（Conductance）

最常用的社区质量度量：

$$\phi(A) = \text{cut}(A) / \min(\text{vol}(A), 2m - \text{vol}(A))$$

- $\text{cut}(A)$ ：从 A 指向外部的边数
- $\text{vol}(A)$ ：A 内所有节点的度数之和

- 分母取最小值：防止偏向极端不平衡的分区

低传导率 = 好社区（内部紧密，外部稀疏）

三、近似个性化 PageRank (Approximate PPR)

3.1 核心思路

从种子节点 s 出发，用 $\text{teleport set} = \{s\}$ 计算 PPR：

- 如果 s 属于一个好的社区，随机游走会被"困在"这个社区里
- 社区内的节点 PPR 值高

3.2 遍历 (Sweep) 算法

1. 计算每个节点相对于种子 s 的 PPR 值
2. 按 PPR 降序排列节点
3. 遍历排序，传导率的局部极小值对应好社区

3.3 Push 操作 (PageRank-Nibble)

高效近似计算 PPR，无需访问整个图：

```
Push(u):  
    r[u] += (1-β) × q[u]      # 将一部分残差累加到结果  
    q[u] = ½ × β × q[u]      # 残差减半  
    对每个邻居 v:  
        q[v] += ½β × q[u] / d(u)  # 另一半残差分配给邻居
```

核心思想：如果节点 u 的残差 $q[u]$ 相对于度数 $d(u)$ 太大 ($q[u]/d(u) \geq \epsilon$)，就把它"push"出去。

3.4 近似PPR 的复杂度

- 时间复杂度： $O(\phi/\log k)$ (ϕ = 传导率, k = 集合大小)
- 只访问与目标集合相关的节点，与图大小无关

四、基于模体的谱聚类 (Motif-based Spectral Clustering)

4.1 模体 (Motif)

模体是网络中重复出现的小型子图模式，代表基本结构单元：

- 三角结构 (Triangle)
- 反馈环 (Feedforward Loop)
- 等

4.2 模体感知的传导率

将边割和体积概念推广到模体：

模体割：需要移除的模体数量
模体体积：所有模体端点数量之和

4.3 算法流程

1. **预处理**：计算每对节点参与模体的次数 $W_{ij}^{(M)}$
2. **近似 PageRank**：基于加权图计算 PPR
3. **遍历**：按降序排列，找到好社区

应用：食物链网络聚类、神经网络结构分析。

五、模块度最大化 (Modularity Maximization)

5.1 模块度 Q

衡量网络划分为社区的整体质量：

$$Q = \sum_c [(\text{实际边数} - \text{期望边数}) / \text{总边数}]$$

- **期望边数**来自配置模型（空模型）：随机重连网络，保持相同度分布
- $Q > 0.3 \sim 0.7$ 表示存在显著的社区结构

5.2 配置模型 (Configuration Model)

构建期望边数的参照网络：

- 相同节点数和总边数
- 但连接随机分布
- 节点 i 和 j 之间期望边数 $= (k_i \times k_j) / 2m$

六、Louvain 算法

6.1 算法特点

- 时间复杂度： $O(n \log n)$
- 支持加权图
- 输出**层次化社区**（树状结构）
- 广泛应用：快速、效果好

6.2 两阶段迭代

阶段1 (Partition) :

- 每个节点最初是一个独立社区
- 遍历节点，尝试将其移入邻居社区
- 计算模块度增量 ΔQ
- 移入产生最大 ΔQ 的邻居社区
- 直到没有移动能增加 Q (局部最优)

阶段2 (Restructuring) :

- 将阶段1发现的社区合并为超级节点
- 构建新网络：超级节点之间边的权重 = 对应社区间所有边权重之和
- 返回阶段1，迭代直到 Q 不再增加

6.3 模块度增益 ΔQ

将节点 i 移入社区 C 的增益：

$$\Delta Q = [\Sigma_{in} + k_{i,in}] / 2m - [\Sigma_{tot} + k_i] / 2m \times [\Sigma_{tot} / 2m]$$

- Σ_{in} : C 内边权重之和
- Σ_{tot} : C 内所有节点度数之和
- $k_{i,in}$: i 与 C 之间边权重

七、L6 知识全景图

社区发现

- ├─ 好社区的定义
 - ├─ 传导率 (Conductance) = $\text{cut}(A) / \min(\text{vol}(A), 2m - \text{vol}(A))$
 - └─ 最小化割边，最大化内部连接
- ├─ 近似PPR
 - ├─ 从种子出发计算PPR
 - ├─ Push操作高效近似
 - └─ Sweep遍历找传导率局部极小
- ├─ 模体聚类
 - ├─ 模体 = 重复出现的小型子图模式
 - └─ 模体感知的传导率
- ├─ 模块度 (Modularity)
 - ├─ 实际边 - 期望边 / 总边数
 - └─ 配置模型计算期望边
- └─ Louvain算法
 - ├─ 贪心策略
 - ├─ 阶段1：划分 (节点移入移出)
 - ├─ 阶段2：重构 (超级节点聚合)
 - └─ $O(n \log n)$

八、真题级要点速记

传导率是社区质量的核心度量——内部连接越多、外部越少，传导率越低。

近似**PPR**的核心思想：从种子出发的随机游走会被"困在"好社区里。

Push操作 只处理残差 $q[u]/d(u) \geq \epsilon$ 的节点，实现与图大小无关的计算复杂度。

Louvain算法 两个阶段不断交替：划分（局部移动找最优）+ 重构（社区聚合到超级节点）。

模块度 Q $> 0.3 \sim 0.7$ 表示存在显著社区结构。

模体聚类 比传统边聚类更能捕捉功能性的结构模式（如食物链中的 Feedback Loop）。